Security - Application Security I - Attacker - Overflows, Underflows and other Vulnerabilities

2026-04-15 00:00
Status: #idea
Tags: Security

Security - Application Security I - Overflows, Underflows and other Vulnerabilities

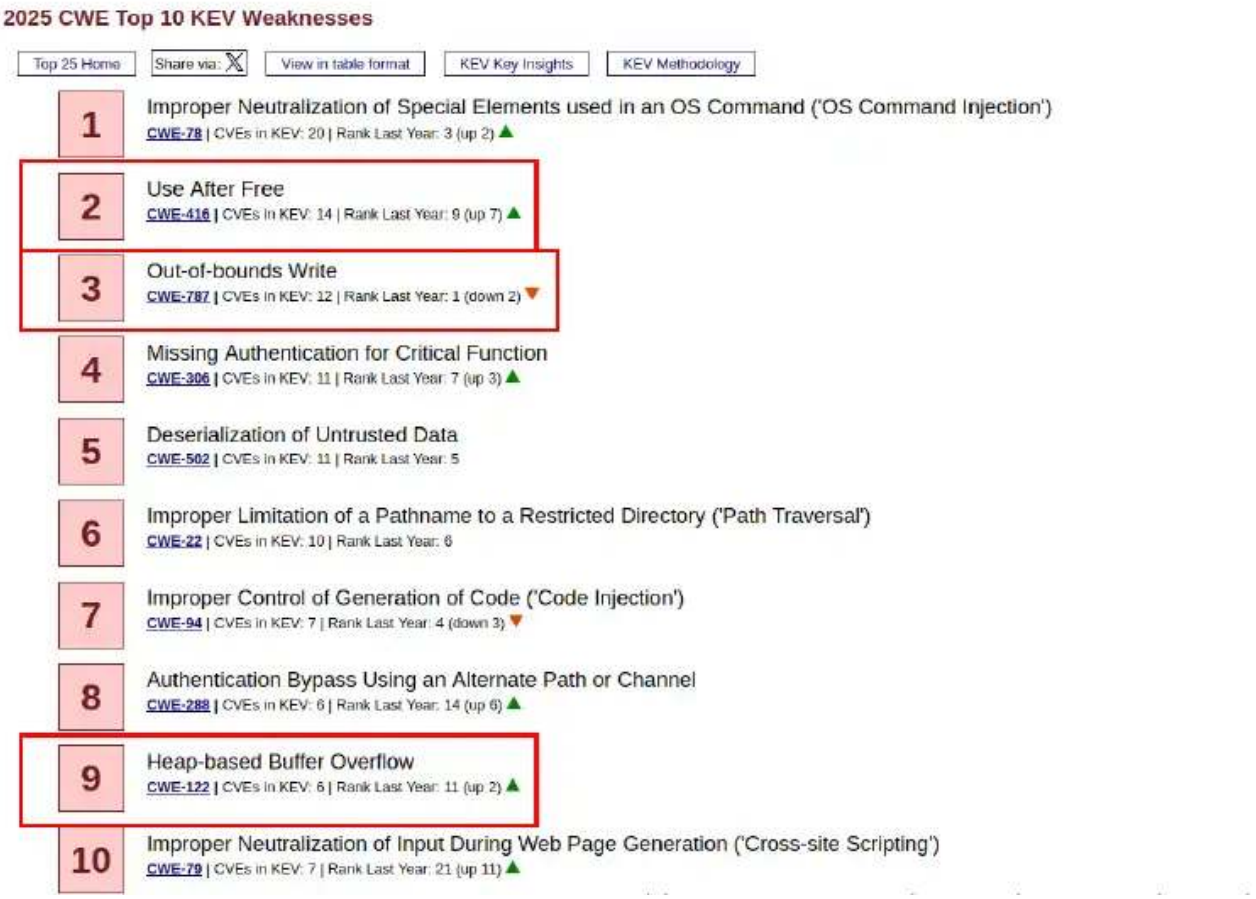
Welcome to class! It’s completely okay that you missed the lecture, and it's great that you already have a foundational understanding of computers. Application Security is all about how software breaks, and more importantly, how attackers force it to break in ways that benefit them.

I am going to walk you through this entire lecture slide by slide, explaining the concepts, the terminology, and the code exactly as if you were sitting in the front row.

Let's dive in.

Part 1: Software Vulnerabilities & The Threat Landscape

(Slides 1 - 3)



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image.webp

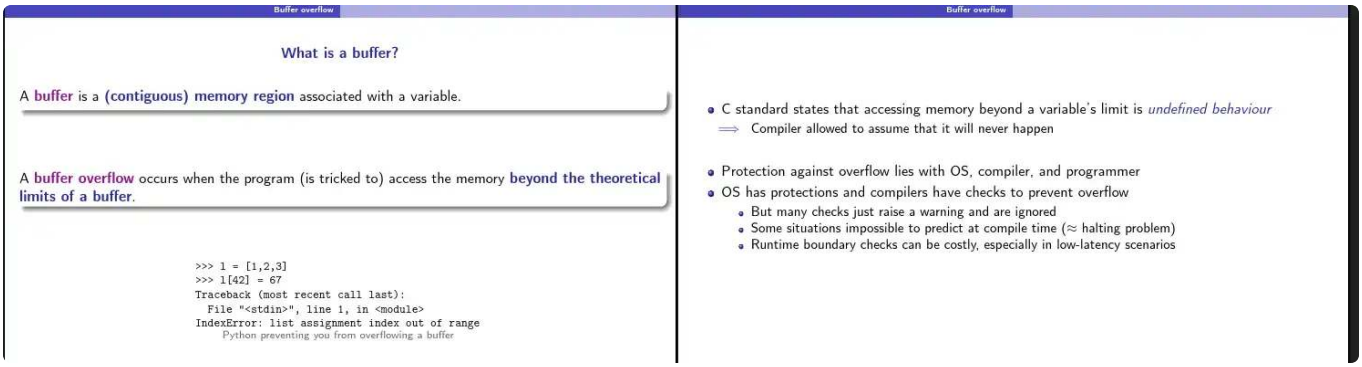
The lecture starts by stating a simple fact: programs run everywhere, and if an attacker can exploit a vulnerability in a program, they gain access to the machine running it. Many of these vulnerabilities come down to how languages like C manage memory.

You asked about **"CWE Top 10 KEV Weaknesses"**.

- **CWE** stands for *Common Weakness Enumeration*. It is essentially a dictionary of all the different ways software can be vulnerable.
- **KEV** stands for *Known Exploited Vulnerabilities*.

So, when the slide shows the "CWE Top 25 KEV Weaknesses," it is showing a "Most Wanted" list of the most dangerous, actively exploited software bugs in the real world right now. As the slide highlights, memory issues like "Out-of-bounds write" (Buffer Overflows) and "Use After Free" are consistently at the top of this list.

What is a Buffer? And Why Does C Allow Overflows?



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-1.webp

A **buffer** is simply a chunk of contiguous (side-by-side) memory reserved for a variable. For example, if you need to store a 10-letter word, you ask the computer for a buffer of 10 bytes.

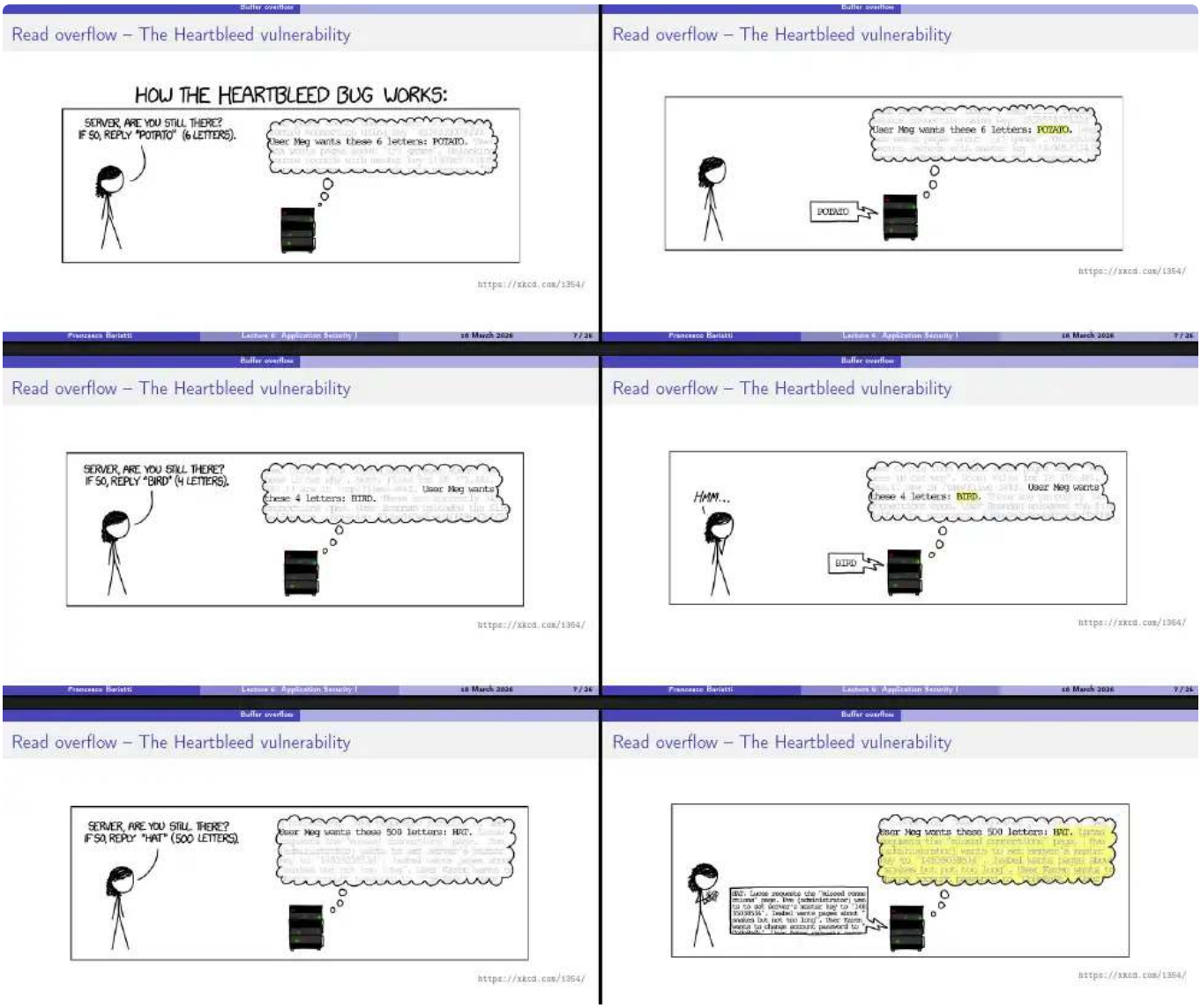
A **buffer overflow** happens when a program is tricked into writing past the physical limits of that buffer, spilling data into adjacent memory.

You asked about **Undefined Behavior** and why the compiler doesn't just stop this:

- **Python vs. C:** If you try to write to the 42nd slot of a 3-slot list in Python, Python actively monitors your program, realizes the mistake, and stops it with an `IndexError`.
- **Undefined Behavior in C:** The C programming language was built to be lightning-fast. The official rules of C state that accessing memory beyond a buffer is "Undefined Behavior." This means the language provides zero guarantees about what will happen. It’s not a defined error; it's just the Wild West.
- **Why the compiler allows it:** Compilers (the programs that translate C code into machine code) are allowed to assume the programmer is perfect. Trying to mathematically predict if a program will *ever* overflow a buffer before it runs is essentially the **Halting Problem**—a famous concept in computer science proving that it's impossible to predict exactly what a complex program will do without actually running it.
- **Why not use runtime checks?** We *could* have C check the length of an array every single time we write to it (like Python does). But doing that check takes processing power. In low-latency environments (like operating systems, high-speed trading, or game engines), those microsecond delays are unacceptable. So, the checks are left out for speed.

Read Overflows & The Heartbleed Bug

(Slides 6 - 7)



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-3.webp

Usually, we think of *writing* too much data as an overflow. But what about *reading* too much? That brings us to **Heartbleed**, one of the most infamous bugs in history.

You asked me to explain the **XKCD Comic**, which perfectly illustrates how this read overflow worked:

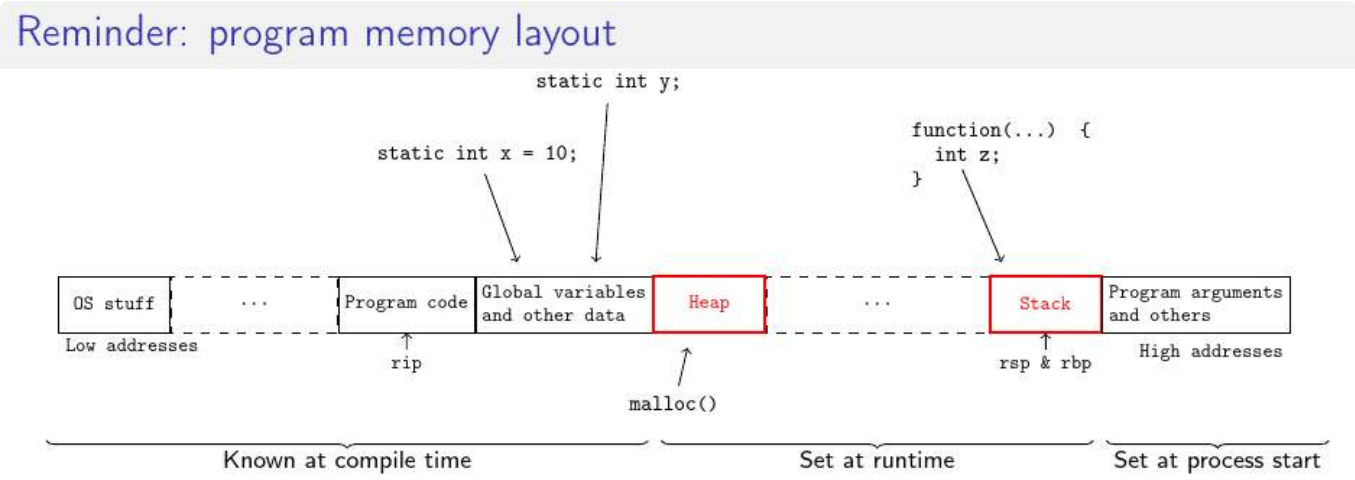
The "Heartbeat" protocol was a feature in OpenSSL (a secure communication library). Its purpose was for a user to ping a server to see if it was still alive.

1. The user sends a word and tells the server how long the word is: *"Reply 'potato' (6 letters)."*
2. The server receives "potato", sees it is 6 letters, and sends it back.
3. **The Exploit:** The attacker lies. The attacker says: *"Reply 'hat' (500 letters)."*
4. The server's code didn't check if "hat" was actually 500 letters. It just grabbed "hat" from memory, and then continued to read the next **497 letters** directly out of its own

internal memory buffer, sending it all back to the attacker!

As the comic shows, those extra 497 letters of memory contained recent requests from other users, master cryptographic keys, and passwords.

The Program Memory Layout

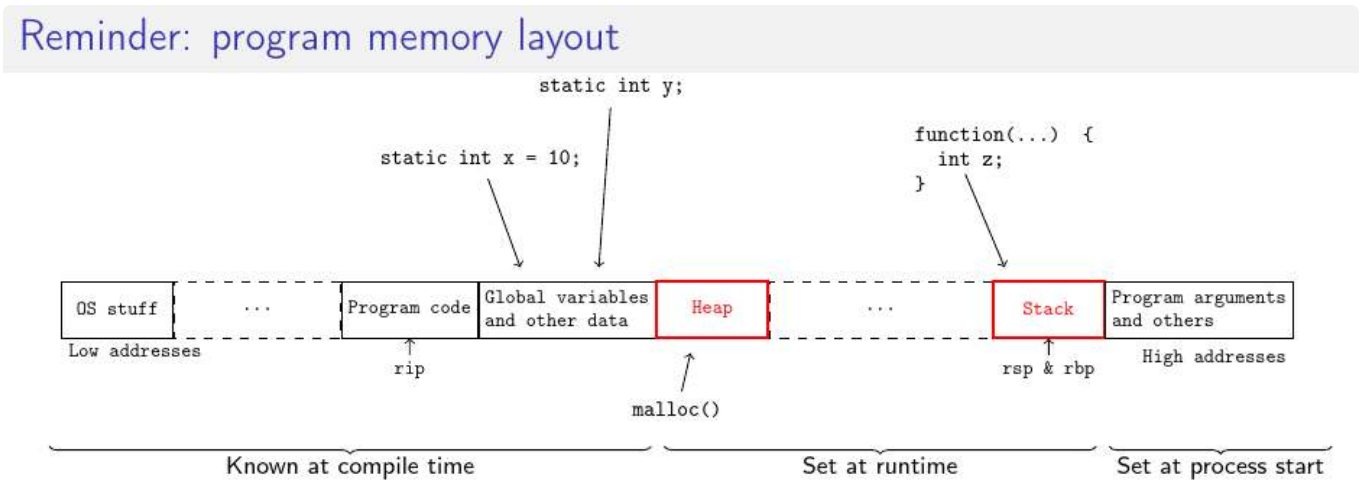


Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-4.webp

Actually a process not a program^ - https://www.youtube.com/watch?v=7ge7u5VUSbE
To understand how to write exploits, we have to understand how memory is laid out when a program runs.

Imagine a tall tower of computer memory:

1. **Program Code (Bottom):** The actual binary instructions the CPU is executing.
2. **Global Variables:** Variables that are available everywhere in the program.
3. **The Heap:** This is dynamic memory. Sometimes you don't know how much memory you need until the program is running (like loading a user's profile picture). You use a function called `malloc()` (Memory Allocate) to grab a chunk of the Heap. The Heap grows *upwards*.
4. **The Stack:** This is temporary, organized memory used for functions. It grows *downwards*.



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-4.webp

This is a fantastic diagram that breaks down the anatomy of a running program. When you execute a program, the operating system gives it a dedicated chunk of memory to use, laid out exactly like this—from low memory addresses at the bottom to high memory addresses at the top.

Let's break down every part of this diagram and answer all of your questions step-by-step.

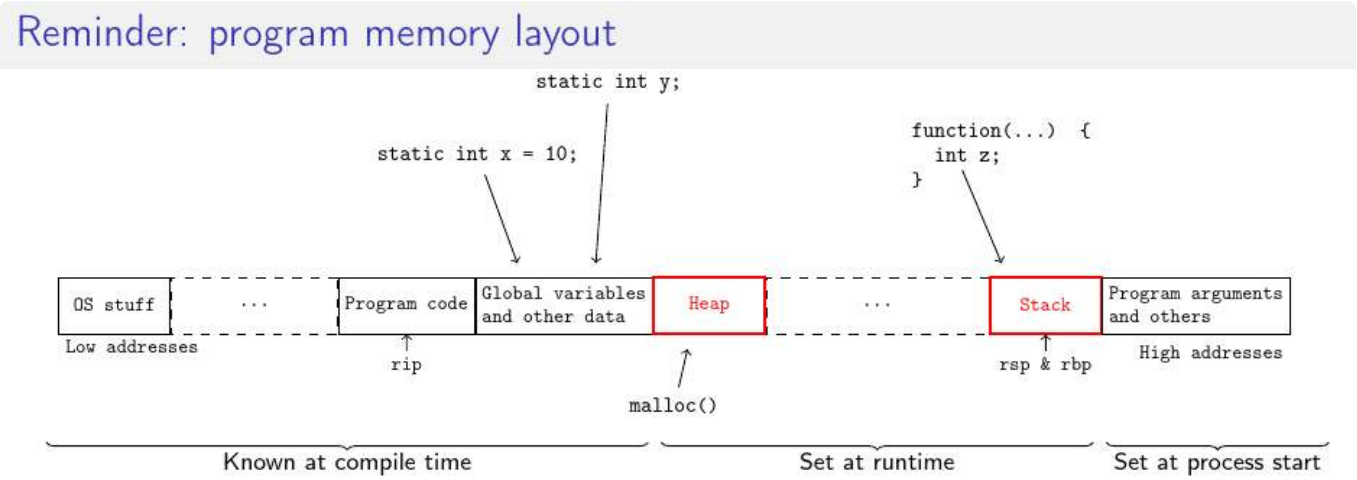
1. Program Code & `rip`

- **What is it?** This is where the actual compiled, 1s-and-0s machine code of your program lives. It is read-only so the program doesn't accidentally overwrite its own instructions.
- **What is `rip`?** `rip` stands for **Instruction Pointer** (specifically for 64-bit Intel/AMD processors). It is a special hardware register inside the CPU that acts like a bookmark. It points to the exact memory address of the *very next instruction* the CPU needs to execute. As your program runs, `rip` constantly moves forward (or jumps around if you have `if` statements or loops) through the Program Code section.

2. Global Variables and Other Data

- **What are they?** A "declared variable" is simply you telling the computer, *"Hey, reserve a box in memory and label it 'score', and make it hold an integer."* * **Local variables** (like `z` in the diagram) live inside specific functions and disappear when the function ends. They are in STACK.
  - **Global variables** (like `static int x = 10;` in the diagram) are declared *outside* of any function. They are available anywhere in your program and exist for the entire time the program is running.
  - **Is it `.data` in assembly?** Yes, exactly!

- The `.data` section holds global variables that are initialized with a specific value (like `x = 10`).
  - The **"other data"** refers to the `.bss` section (which holds global variables that are uninitialized or set to zero, like `static int y;`) and the `.rodata` section (read-only data, like hardcoded text strings such as `"Hello World"`).
- **Why is everything up to here "Known at compile time"?**
  Before your program ever runs, the compiler reads your code and knows exactly how many instructions there are, how many global variables exist, and how much space they require. Because this is fixed and unchanging, the compiler bakes the exact size of these sections directly into the executable file.



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-4.webp

3. The Heap & `malloc()`

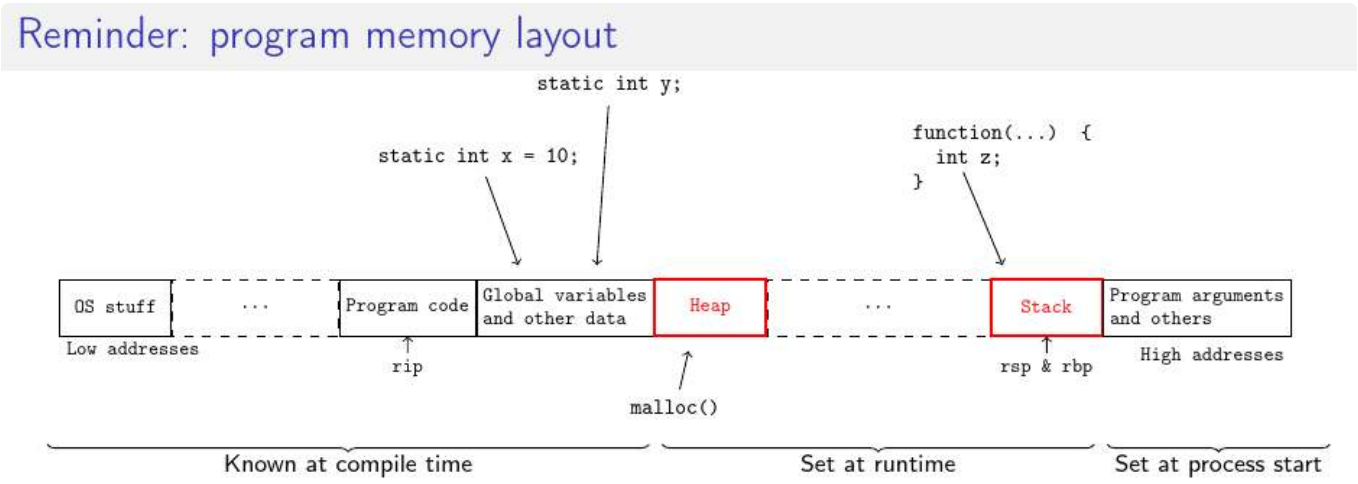
- **What is the Heap?** The Heap is a giant pool of free memory used for **dynamic memory allocation**. You use the heap when you don't know how much memory you will need until the program is actually running.
- **Is it for instances of classes/objects?** Yes! In object-oriented languages like Java, Python, or C#, whenever you create a new object (e.g., `new Player()`), that object is placed exactly here, on the Heap.
- **What is `malloc()`?** `malloc` stands for **Memory Allocation**. In the C programming language, you don't have a `new` keyword. Instead, you call the `malloc()` function to politely ask the operating system: *"Can I please have X amount of bytes from the Heap?"*
- **Example:** Imagine you are writing a program that loads an image, but you don't know how big the image is until the user selects it.

```
// The user selects a 1000-byte image at runtime.
// We ask malloc to reserve 1000 bytes on the Heap.
char* image_data = malloc(1000);
```

The Heap starts at a lower address and grows *upwards* as you allocate more memory.

4. The Stack, `rsp`, and `rbp`

- **What is the Stack?** The stack is a highly organized, temporary scratchpad used for running functions. Whenever a function is called, a new block of memory (called a "stack frame") is created to hold that function's local variables. When the function finishes, that memory is instantly freed up.
- **Why does the function point to it?** In the diagram, there is a function containing `int z;`. `z` is a local variable. The compiler knows that `z` only needs to exist while that specific function is running, so it places `z` on the Stack, not in global memory.
- Add by aryan - I think its because functions are like objects on the stack too, think about how you can simluate recursion with a stack, a call stack, so you add the function() to stack, then when you encounter a new function thats added to the stack and you can only get back to the previous call when this one finishes.
- **What are `rsp` and `rbp`?** These are CPU registers used to keep track of the stack.
  - `rsp` **(Stack Pointer):** Points to the very "top" of the stack (though visually in this diagram, the stack grows *downwards* towards lower addresses, so it points to the lowest address the stack is currently using).
    - `rbp` **(Base Pointer):** Points to the start (or base) of the current function's stack frame. This helps the program know exactly where the current function's local variables begin.



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-4.webp

5. Why are the Heap and Stack "Set at runtime"?

The compiler cannot predict the future. It doesn't know if the user will click a button that triggers a function 5 times or 500 times (which affects the Stack). It doesn't know if the user will load a 1MB file or a 100MB file (which affects the Heap). Therefore, the sizes of

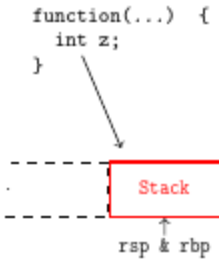
the Heap and Stack are flexible and change continuously while the program is actively running.

(Note: As the diagram shows, they grow towards each other. If they ever collide, your program crashes with an "Out of Memory" or "Stack Overflow" error!)

6. Program Arguments and Others

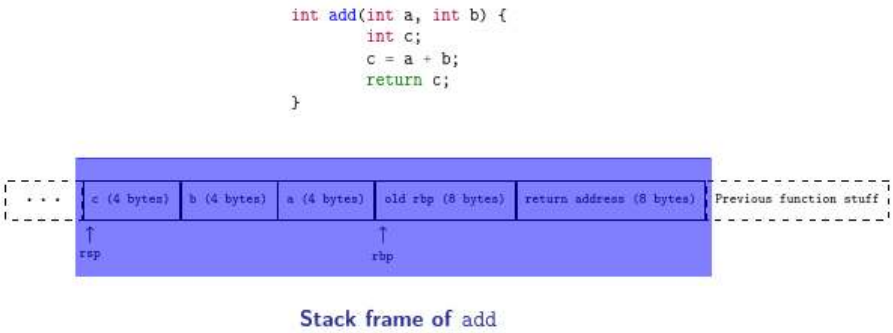
- **What are program arguments?** When you run a program from the command line, you often pass it arguments. For example, if you type `ping google.com` in your terminal, `ping` is the program, and `google.com` is the argument.
- **"Others"** refers to "Environment Variables"—hidden settings on your computer (like your username or system paths) that the program might need to read.
- **Why "Set at process start"?** These values are provided by the operating system the exact moment you hit "Enter" to launch the program. They are injected into the very highest memory addresses before the first line of your `Program Code` is ever executed. Once the program starts, these arguments generally don't change size.

Inside the Stack Memory



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-5.webp

Inside a function: stack memory



Stack frame of add

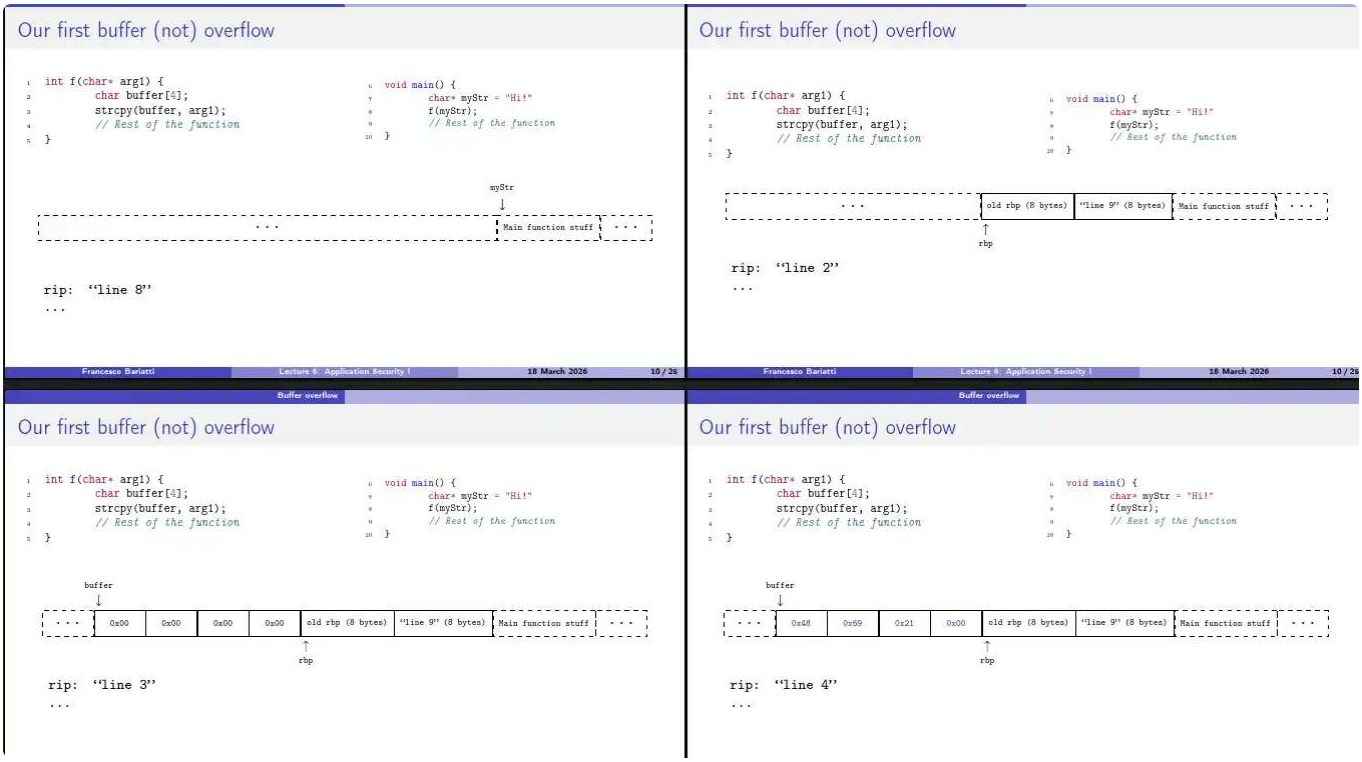
Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-6.webp

The Stack works exactly like a stack of plates. When a function is called, a "Stack Frame" (a new plate) is placed on top. When the function finishes, its plate is removed.

You asked about **RSP, RBP, and Return Addresses**:

- **RSP (Stack Pointer):** A CPU register that always points to the very top of the stack.
- **RBP (Base Pointer):** A CPU register that points to the bottom (base) of the current function's workspace.
- **Return Address:** This is the most critical concept. When a function (like `add()`) finishes, (so it's cleared everything from the left of ret addr) the CPU needs to know where to go back to in the main program (or the function that called this function). When the function is called, the CPU saves the "Return Address" on the stack. **If an attacker can overwrite this return address, they control the CPU.**

Our First Buffer (Not) Overflow vs. The Exploit



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-7.webp

Let's look at the code provided in the slides.

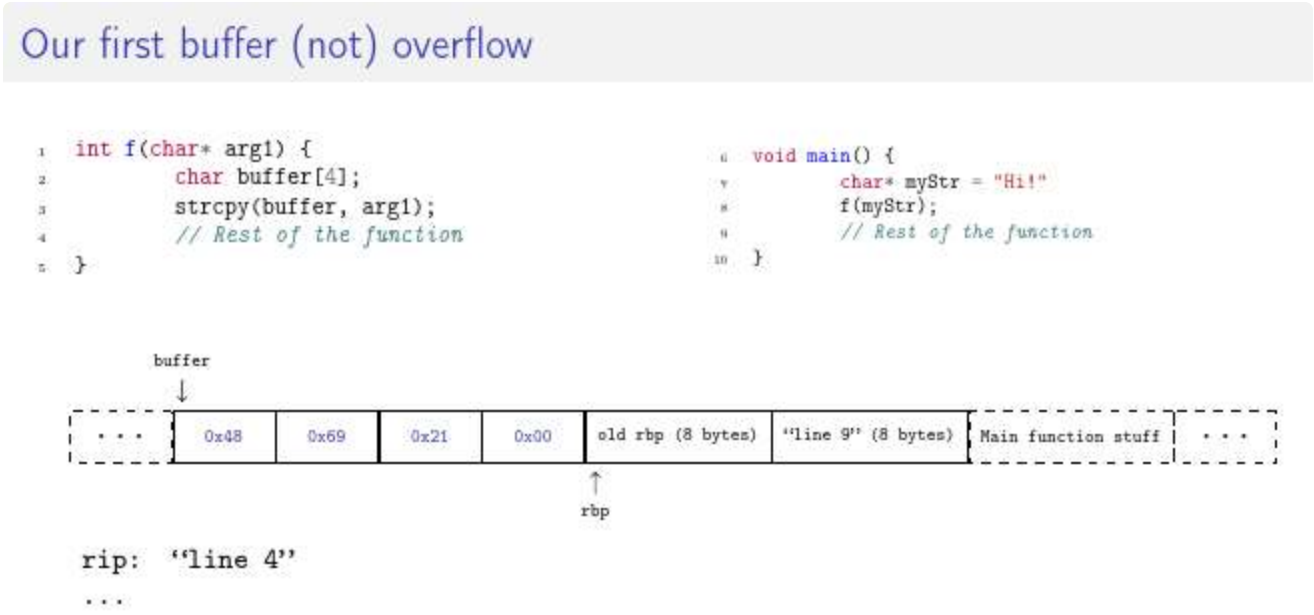
```
int f(char* arg1) {
    char buffer[4];
    strcpy(buffer, arg1);
    // Rest of the function
}

void main() {
    char* myStr = "Hi!"; // Later changed to "I have hacked you!!"
```

```
    f(myStr);
}
```

You asked what `rip` means. `RIP` is the **Instruction Pointer**. It is a CPU register that simply holds the memory address of the *current line of code* the computer is executing.

Scenario 1: Not Overflowing

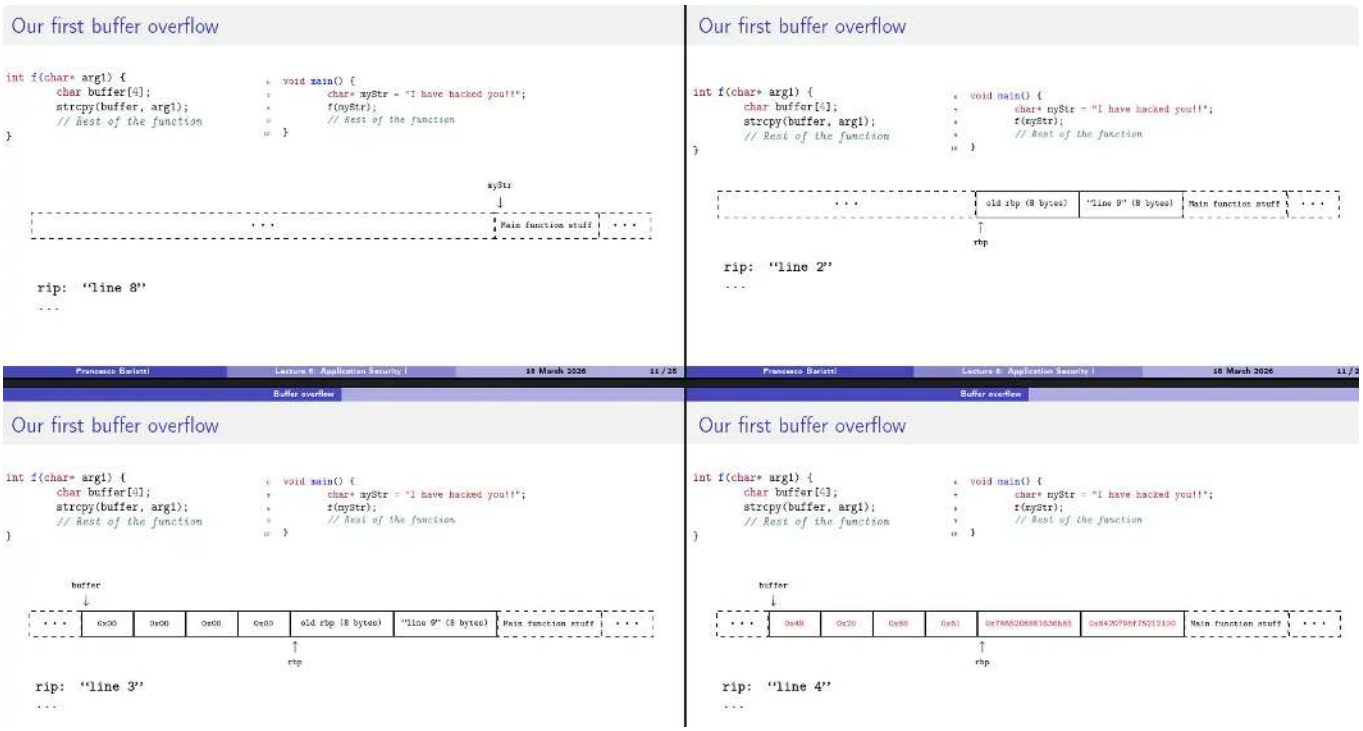
`myStr` is "Hi!". That is 3 letters plus an invisible `\0` (null byte) that marks the end of a string in C. It fits perfectly into `buffer[4]`. The function copies it, finishes, looks at the Return Address saved on the stack, and safely jumps back to `main()`. Look at the 4 slides above. This is the final state:



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-8.webp

First RIP: line 8, myStr is stored in the function local vars of main. Then we run the function f() so we add the ret adrr to stack (line9) and the old rbp (the bottom of main's stack), and after that we start the function f, so lets place the rbp here. Then rip: line 2, we start 4 bytes as a buffer on the stack. Okay now let's copy the arg ("Hi!") into the buffer with strcpy. (BTW this is done i think by going to our own RBP, 8 bytes under that is the old RBP, and parameters always come from the previous call so thats where we find the parameter.)

Scenario 2: Overflowing

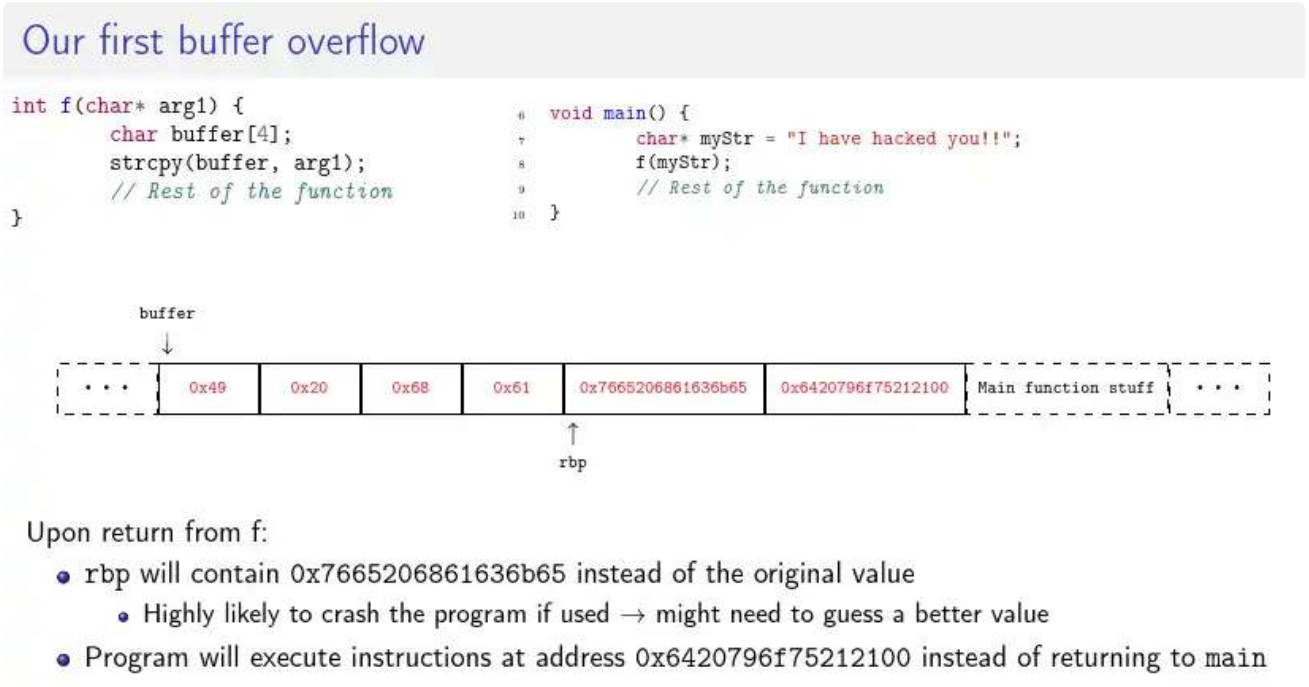


Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-9.webp

`myStr` is now "I have hacked you!!". This is way larger than 4 bytes.

The function `strcpy()` (string copy) is a notoriously dangerous C function because it *does not check the size of the destination buffer*. It just starts copying letters.

1. It fills the 4 bytes of `buffer`.
2. It keeps going, spilling upwards into the stack memory.
3. It overwrites the `old rbp`.
4. Crucially, **it overwrites the Return Address.**

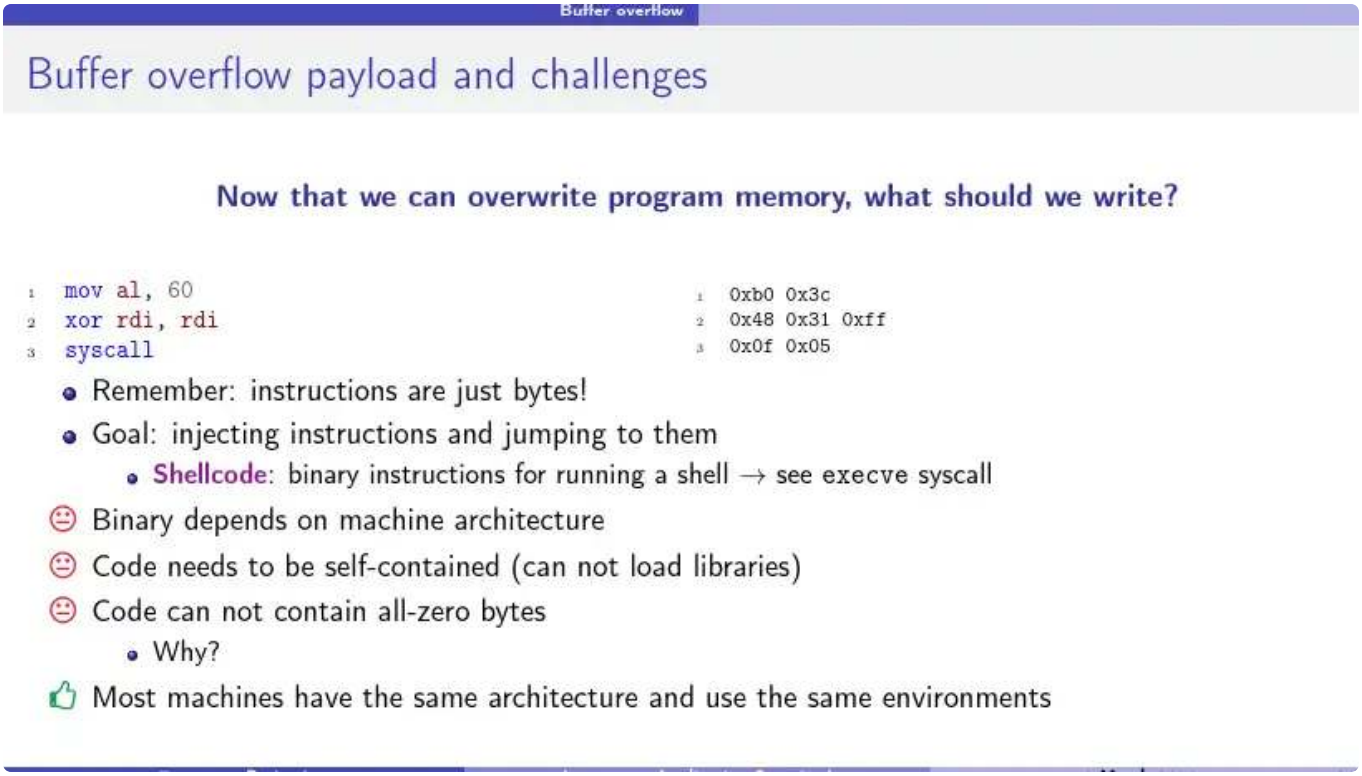


Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-10.webp

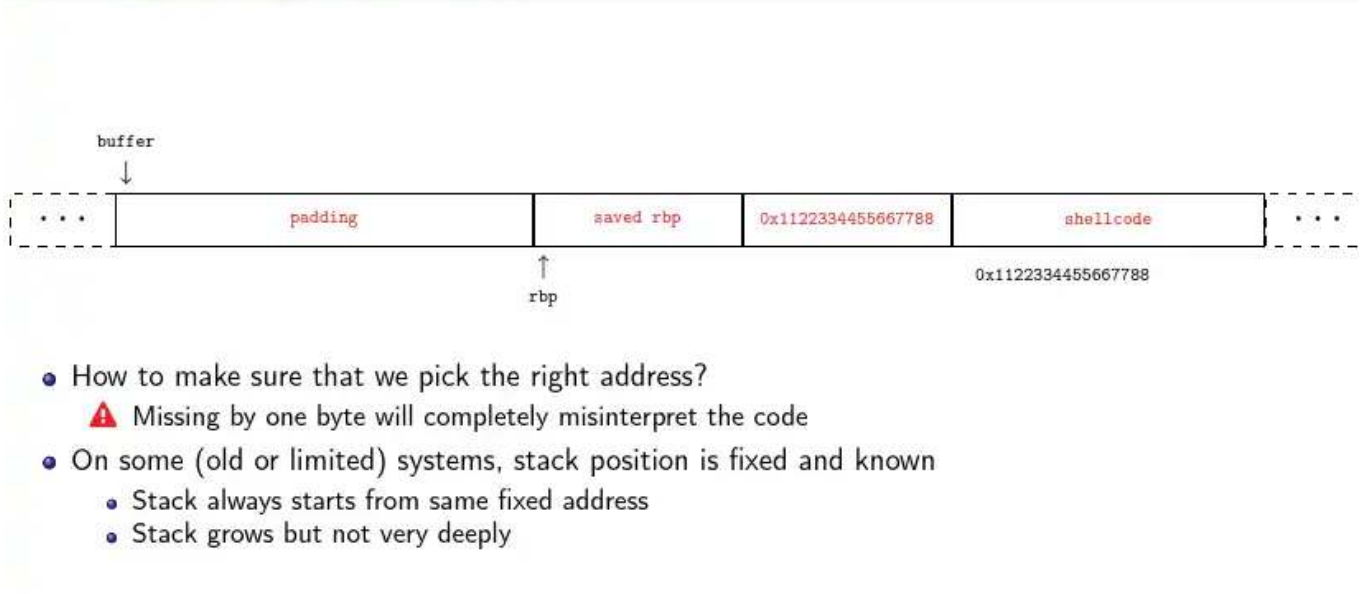
When the function `f()` finishes, the CPU looks at the stack to find the Return Address. Instead of a valid memory location, it finds the hex representation of the letters `"ed you!!"`. The CPU attempts to jump to that random address, gets confused, and crashes the program (this is a Segmentation Fault).

See how you can abuse this?

Taking Control: Shellcode, Payloads, and NOP Sleds



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-11.webp



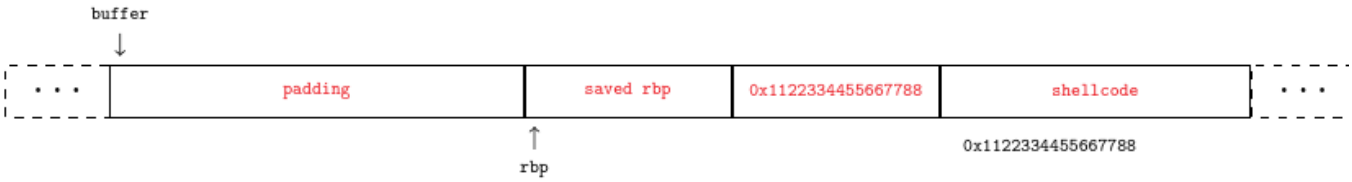
Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-12.webp

Crashing a program is fun, but controlling it is the goal. Instead of overflowing the buffer with garbage text, we overflow it with actual machine instructions (Assembly code).

You asked about **Syscalls, Shellcode, and Zero Bytes:**

- **Shellcode:** This is a tiny, pre-compiled piece of binary code that attackers inject. The goal of this code is usually to open a "shell" (a terminal window) so the attacker can type commands into the server.
- **Syscall (`execve`):** A syscall is a "System Call". It is how a program politely asks the Operating System to do something. Our shellcode uses the `execve` syscall to ask the OS to execute a command-line shell.
- **Why no zero bytes?** In C, a zero byte (`0x00`) literally means "End of String". If your injected shellcode contains a zero byte, the `strcpy()` function will think it reached the end of your text and stop copying immediately, cutting your malicious payload in half! Attackers have to write clever assembly code that completely avoids using zeros.

Guessing the Address & The NOP Sled:

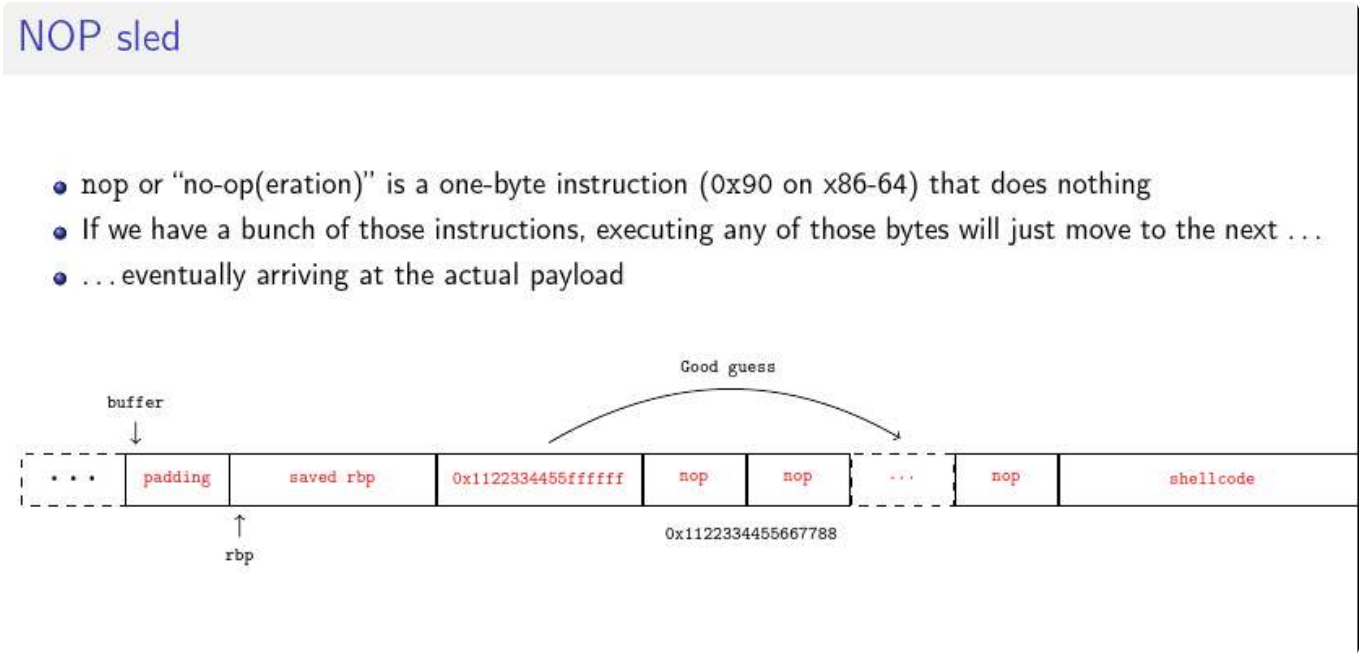


Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-13.webp

To make the CPU run our shellcode, we must overwrite the Return Address with the exact

memory address of where our shellcode landed on the stack. If we miss by even *one byte*, the CPU will misinterpret the instructions and crash.

Since guessing exact memory addresses is hard, attackers use a **NOP Sled**.

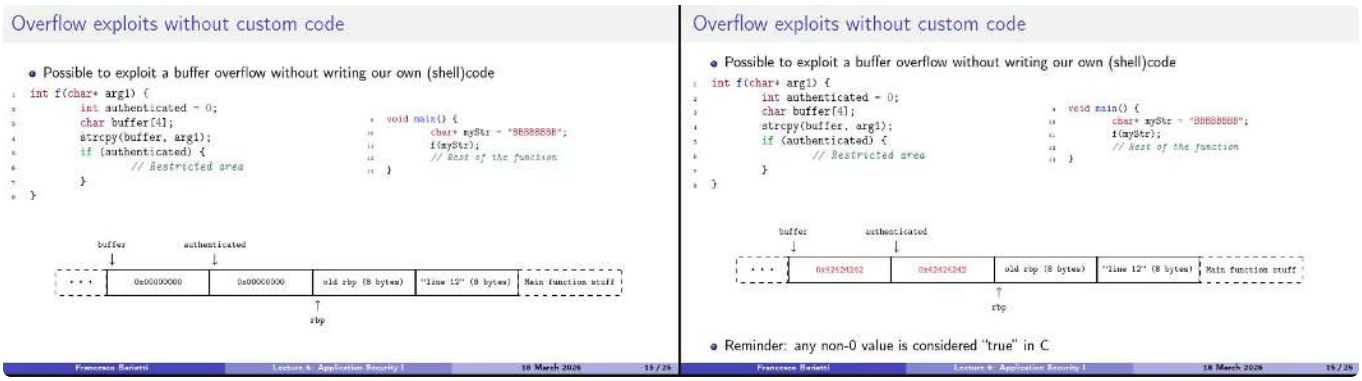


Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-14.webp

- **NOP:** Stands for "No Operation". It's a CPU instruction that literally does nothing except tell the CPU to move to the next instruction.
- By packing the buffer with hundreds of NOPs *before* the shellcode, the attacker creates a massive target. If they overwrite the return address to point *anywhere* in the NOPs, the CPU will land there, "slide" down the sled doing nothing, and crash right into the shellcode at the bottom.

Very smart! That's clever asf

Buffer Overflows Without Custom Code



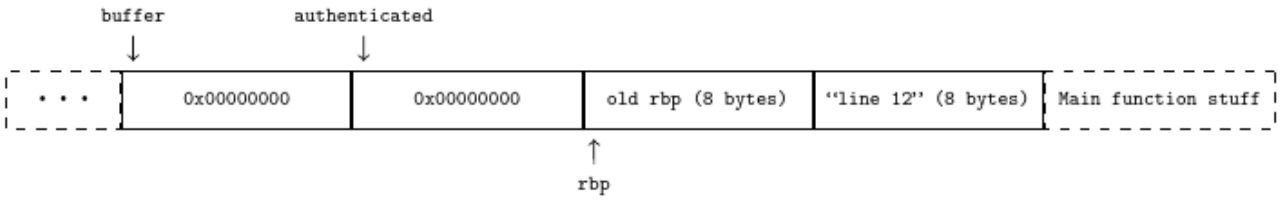
Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-15.webp

(Slide 15)

Sometimes, you don't even need shellcode. Look at this code:

```
int f(char* arg1) {
    int authenticated = 0;
    char buffer[4];
    strcpy(buffer, arg1);
    if (authenticated) {
        // Restricted area
    }
}
void main() {
    char* myStr = "BBBBBBBB";
    f(myStr);
}
```

Because `authenticated` is declared right next to `buffer` on the stack, overflowing `buffer` spills directly into `authenticated`. The string "BBBBBBBB" overflows the 4-byte buffer, and the remaining four 'B's overwrite the `authenticated` integer (changing it from `0` to `0x42424242`, which is the hex value of BBBB).

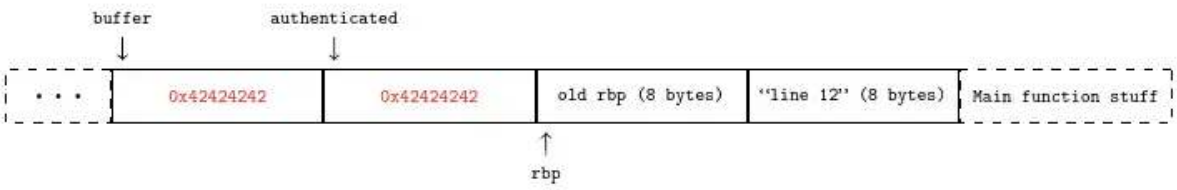


Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-17.webp

- Possible to exploit a buffer overflow without writing our own (shell)code

```
1  int f(char* arg1) {
2          int authenticated = 0;
3          char buffer[4];
4          strcpy(buffer, arg1);
5          if (authenticated) {
6                  // Restricted area
7          }
8  }
```

```
9  void main() {
10         char* myStr = "BBBBBBBB";
11         f(myStr);
12         // Rest of the function
13 }
```



- Reminder: any non-0 value is considered "true" in C

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-16.webp

In C, *any* value that is not 0 is considered "True". The program sees `authenticated` is now `0x42424242`, assumes it is True, and lets you into the restricted area!

Heap Overflows

Heap overflow

- Until now, we have been doing **"stack smashing"**
- ...but programs use another memory region: the **heap**!
  - Memory reserved via `malloc`
  - Memory used to survive beyond a function's end
  - Many high-level languages (Java, Python) use the heap a lot for object-oriented programming

```
void f() {
        char* buf = (char*) malloc(4);
        char* buf2 = (char*) malloc(42);
        char* user_input = /*read user input*/
        strcpy(buf,user_input);
        // Rest of the function
}
```

- If user input > 4, then overwrite heap
- It might be buf2, it might be something unrelated!

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-18.webp

As mentioned earlier, the Heap is dynamic memory used by `malloc()`.

```
void f() {
    char* buf = (char*) malloc(4);
    char* buf2 = (char*) malloc(42);
    char* user_input = /*read user input*/
    strcpy(buf,user_input);
}
```

If we overflow `buf` here, we aren't on the stack anymore, so there is no Return Address to overwrite. Instead, we spill into whatever data is sitting next to us on the heap (like `buf2`, or sensitive object metadata in languages like Java or C++). By corrupting this data, we can change the internal logic of the program.
So consequences:

- Overflow into adjacent objects
- In object-oriented programming: overflow into vtables (class information)
  $\Longrightarrow$ Change object methods!
- Overflow heap metadata itself

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-19.webp

---

Part 2: Integer Overflows & Underflows

(Slides 18 - 19)

Computers store numbers in a fixed amount of space (usually 32 bits for an integer).

Integer Overflow:

```
1  char** read_network_packet() {
2          packet* p = wait_for_packet();
3          int n_strings = p->header->n_strings;
4          char** strings = (char**) malloc(n_strings * sizeof(char*));
5          for (int i = 0; i < n_strings; i++)
6                  strings[i] = p->get_string(i);
7          return strings;
8  }
```

What is the problem?

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-20.webp

Line 1: `char** read_network_packet() {` This starts a function. Its job is to read data coming over a network (like the internet) and return a list of text strings (`char**` is C jargon for "a list of text strings").

Line 2: `packet* p = wait_for_packet();` The program pauses and waits for a message (a "packet") to arrive from the network. When it arrives, it saves it in a variable named `p`.

Line 3: `int n_strings = p->header->n_strings;` The program looks at the packet's "header" (like the label on a shipping box) to see how many text strings are inside. It saves this number in an integer variable called `n_strings`.

Line 4: `char** strings = (char**) malloc(n_strings * sizeof(char*));` Here is where we ask for memory on the **Heap** (which you learned about in the previous diagram!). We use `malloc()` to say: *"Hey computer, I need space. Please multiply the number of strings*

(`n_strings`) by the size of a single string slot (`sizeof(char*)`), and reserve that much total memory for me."

Lines 5 & 6: `for (int i = 0; i < n_strings; i++)` `strings[i] = p->get_string(i);` This is a loop. It says: *"Count from zero up to the total number of strings. One by one, pull the string out of the network packet and place it into the memory space we just reserved."*

Line 7: `return strings;` Hand the fully loaded list of strings back to whatever part of the program asked for it.

```
char** read_network_packet() {
    packet* p = wait_for_packet();
    int n_strings = p->header->n_strings;
    char** strings = (char**) malloc(n_strings * sizeof(char*));
    // ...
```

If an attacker sends a packet claiming it has `536,870,912` strings, the program multiplies that by 8 (the size of a pointer). That equals `4,294,967,296`, which is larger than the maximum size a standard 32-bit integer (malloc(4294967296)) can hold! Just like an old car odometer rolling past 999,999 back to 0, the integer wraps around to `0`. `malloc(0)` allocates zero memory, but the loop below it tries to write millions of strings, causing a massive memory corruption.

- n_string received over the network → unsanitized user input
- What if n_string = 536870912?
  - 536870912 * sizeof(char*) = 536870912 * 8 = 4294967296
  - More than the max value in an int!
- 536870912 * sizeof(char*) **overflows to 0**
- malloc allocates 0 bytes: **all writes overflow**

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-21.webp

Integer Underflow:

```
1  void process_input(const char *input, unsigned int length) {
2          char buffer[100];
3          // We only have space for 99 characters plus null terminator
4          if (length - 1 > 100) {
5                  printf("Input too long!\n");
6                  return;
7          }
8          // Rest of function
9  }
```

What is the problem?

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-22.webp

1. What do the keywords mean?

- `unsigned` : Your guess is exactly right! Normally, integers (`int`) use one bit of memory t store the "sign" (whether the number is positive or negative). An `unsigned int` drops the negative sign entirely. It can **only be positive or zero**. Because it frees up that sign bit, an `unsigned int` can hold a maximum positive number twice as large as a normal `int` .
- `const` : This is short for "constant." When put in front of `char *input` , it acts as a safet promise. It tells the compiler: *"This function is allowed to read the `input` text, but it is strictly forbidden from changing or modifying it."*

---

2. Concise Code Explanation

Here is what the code is trying to do line-by-line:

1. A function named `process_input` receives some text (`input`) and the number of characters in that text (`length`).
2. It creates a temporary local box called `buffer` that can hold exactly 100 characters.
3. It tries to perform a safety check: "If the length of the input minus 1 is greater than 100, the input is too big."
4. If the text is too big, it prints "Input too long!" and stops. Otherwise, it proceeds to process the text.

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-23.webp

```
void process_input(const char *input, unsigned int length) {
    char buffer[100];
    if (length - 1 > 100) {
        printf("Input too long!\n");
        return;
    }
    // ...
```

Here, `length` is an *unsigned* integer, meaning it cannot be a negative number. If an attacker provides an input of length `0`, the math becomes `0 - 1`. Since it can't be -1, it underflows, wrapping backwards to the absolute maximum possible value (like 4 billion). 4 billion is indeed greater than 100, bypassing the security check while the actual length is 0.

- If `length = 0`, then `length -1 = MAX_INT`
- **Will pass check** while conceptually being wrong

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-24.webp

subtracting from an unsigned int -> dangerous

Buffer overflow summary

A **buffer** is a **(contiguous) memory region** associated with a variable.

A **buffer overflow** occurs when the program (is tricked to) access the memory **beyond the theoretical limits of a buffer**.

Dangers:

- Code injection
- Modify program's state
- Read restricted information

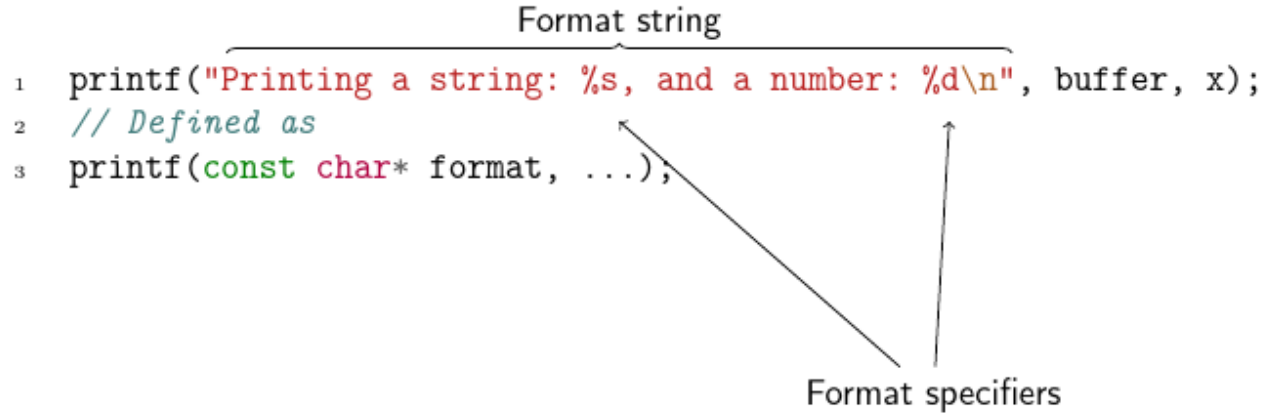
Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-25.webp

---

Part 3: String Formatting Vulnerabilities

(Slides 20 - 23)

The `printf` function in C is used to print strings, but it uses "Format Specifiers" (like `%s` for string, or `%d` for integer) as placeholders for variables.



Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-26.webp

```
void safe() {
        char buf[80];
        // Assume this is safe
        get_user_input(buf);
        printf("User entered:\n",buf);
        printf("%s",buf);
}
```

```
1  void vulnerable() {
2          char buf[80];
3          // Assume this is safe
4          get_user_input(buf);
5          printf("User entered:\n",buf);
6          printf(buf);
7  }
```

What is the problem with the vulnerable function?

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-27.webp

```c
void vulnerable() {
    char buf[80];
    get_user_input(buf);
    printf(buf); // VERY DANGEROUS
}
```

The programmer assumes `printf(buf)` will just print what the user typed. But what if the user types `100% devilish %s`?

Because there is a `%s`, `printf` thinks, "Ah, I need to look for a variable to print!" Since the programmer didn't provide one, `printf` just grabs whatever happens to be sitting next to it on the stack and prints it. This allows an attacker to leak sensitive data from memory just by typing `%s` or `%x`.

- **User can inject their own format**
- `printf` assumes that it is called with enough arguments
- It will happily print whatever happens to be where arguments normally are

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-28.webp

Even worse, the `%n` format specifier actually *writes* the number of characters printed so far into a memory address. So, using `printf`, an attacker can literally overwrite memory!

- `printf("100% devilish");` → Ignore spaces: prints the first "argument" as integer (%d)
- `printf("%s");` → Prints bytes at address given by first "argument"
- `printf("100% no way!");` → **Writes** to address given by first "argument"

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-29.webp

This is one of the sneakiest and most fascinating bugs in C programming! It's called a **Format String Vulnerability**.

To understand how this is misused, you have to understand one fatal flaw about how `printf` works: **`printf` blindly trusts the string.** It doesn't check if you actually passed variables to it; it just counts the `%` signs and starts blindly ripping data off the stack (the temporary memory where variables live).

1. Why are spaces ignored?

Look closely at the text in the examples: `100% devilish` and `100% no way!`. Notice the space after the `%` sign?

In C, a space is actually a **valid formatting flag**.

- Normally, you use `%d` to print a number.
- If you use `% d` (with a space), it tells `printf`: *"Print this number, and if it is positive, put a blank space in front of it so it aligns nicely with negative numbers."*

So, when the user types `100% devilish`, `printf` scans the text, sees the `% d`, and thinks: *"Ah! A format specifier! I need to go grab an integer from memory and print it here."* It completely consumes the `%` and the `d`, leaving the rest of the text mangled.

- `printf("100% devilish");` → Ignore spaces: prints the first "argument" as integer (%d)
- `printf("%s");` → Prints bytes at address given by first "argument"
- `printf("100% no way!");` → **Writes** to address given by first "argument"

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-29.webp

2. How the Attacks Work (The Examples)

When a programmer writes `printf(buf);`, they are handing the steering wheel directly to the user. If the user types formatting characters, `printf` will execute them.

Here is how an attacker weaponizes the three examples at the bottom of the image:

- **Leaking Stack Memory (`%d` or `%x`)**
  - **What they type:** `%x %x %x %x`
    - **What happens:** `printf` sees four `%x` (hexadecimal) specifiers. Because the programmer didn't actually provide four variables, `printf` just grabs the next four chunks of raw data sitting on the memory stack and prints them to the screen.

    - **The Misuse:** An attacker can spam `%x` to dump hundreds of lines of the program's raw memory. This can reveal secret passwords, encryption keys, or memory addresses needed for further attacks.
- **Reading Arbitrary Memory (`%s`)**
- **What they type:** `%s`
    - **What happens:** `%s` is meant to print a string. It tells `printf`: *"Grab the next value from the stack, treat that value as a memory address, go to that address, and print whatever text is there."*
    - **The Misuse:** If an attacker can manipulate the stack slightly, they can force `printf` to look at a *specific* memory address (like where an admin password is stored) and print it back to them. If it points to an invalid address, the program instantly crashes.
- **Overwriting Memory (`%n`)**
- **What they type:** `100% no way!` (which triggers `% n`)
    - **What happens:** `%n` is incredibly dangerous. Instead of reading data, it **writes** data. It counts how many characters have been printed so far, grabs the next value on the stack (treating it as a memory address), and *writes that count into that memory address*.
    - **The Misuse:** An attacker can use this to rewrite the program's variables while it's running! By carefully padding the output with blank spaces before the `%n`, they can control the exact number that gets written. They can use this to overwrite security checks (changing an `is_admin = 0` to `is_admin = 1`) or hijack the program entirely.

3. How to fix it (The Right Way)

The fix is incredibly simple. You must never let user input act as the `format` argument.

The Dangerous Way:

```
// The user controls the format string
printf(user_input);
```

The Safe Way:

```
// We explicitly tell printf that the input is strictly text (%s)
printf("%s", user_input);
```

By explicitly providing `"%s"` as the format string, `printf` knows exactly what to do. It will treat `100% devilish` as pure, harmless text and print it exactly as the user typed it,

ignoring any rogue `%` signs inside the user's input.

Part 4: TOCTOU (Time-Of-Check Time-Of-Use)

(Slide 24)

Finally, we have a vulnerability that has nothing to do with memory, but rather *time*. This is a type of Race Condition.

```
1  if(can_access("file.txt") == 0) {
2          printf("Unauthorized!\n");
3          exit(1);
4  }
5  fd = open("file.txt", /*options*/);
6  write(fd, /*content*/);
```

What is the problem?

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-30.webp

```c
1  if(can_access("file.txt") == 0) {
2      printf("Unauthorized!\n");
3      exit(1);
4  }
5  fd = open("file.txt", /*options*/ );
6  write(fd, /*content*/ );
```

The program checks if you are allowed to access `file.txt` (Line 1). If you are, it proceeds to open it (Line 5).

However, CPUs are fast, but they multitask. There is a tiny fraction of a microsecond between the check at Line 1 and the opening at Line 5. If an attacker runs a rapid-fire script that swaps out `file.txt` with a shortcut to the system's master password file exactly in between those two lines, the program will open the password file! The "Check" happened on the safe file, but the "Use" happened on the malicious file.

- **What if attacker changes meaning of `file.txt` on disk between line 2 and 6?**
  - e.g. symlink `file.txt` → `/etc/passwd`
- This is a **Time-Of-Check Time-Of-Use** vulnerability (TOCTOU)
  - Previous result of a check is used, but that result should have changed
  - A type of **race condition**

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-31.webp

A **race condition** occurs when a system's behavior depends on the exact, unpredictable timing of events—like two distinct processes "racing" to access or modify shared resources at the same time.

In the context of the image you provided (a **TOCTOU** or Time-Of-Check Time-Of-Use vulnerability), here is exactly how that race plays out:

1. **The Check (Line 1):** The program asks the operating system, *"Am I allowed to write to* `file.txt`*?"* The OS says *"Yes, it's a safe file."*
2. **The Window of Opportunity:** The computer takes a tiny fraction of a second to move from the check to the actual opening of the file.
3. **The Race:** During that microscopic delay, an attacker "races" the program. They rapidly delete `file.txt` and replace it with a shortcut (a symlink) pointing to a critical system file, like the master password file (`/etc/passwd`).
4. **The Use (Lines 5-6):** The program reaches the code to open and write to the file. Blindly trusting the check it did a millisecond ago, it writes its data.

The Result: If the attacker wins the race, the program is tricked into overwriting a highly sensitive system file using its authorized permissions, completely compromising the system. The vulnerability exists because the programmer assumed the state of the world couldn't change between the "Check" and the "Use."

Supply-chain vulnerabilities

https://www.crowdstrike.com/en-gb/cybersecurity-101/exposure-management/log4j-vulnerability/

Security - Application Security I - Overflows, Underflows and other Vulnerabilities - image-32.webp

That covers the entirety of Lecture 6! Let me know if you want to dig deeper into how the GDB debugger looks at these memory addresses, or if any of the C logic is still bugging you.

References

https://gemini.google.com/app/0b36b9bc69fa4646
chap 6
lec6 - Applciation Secuirty I